



OpsGate

DailyOps.Tech

OpsGate - Installation & deployment client

[Integrateurs](#) · [Admin systeme](#) · [Pilote DSI/RSSI](#)

[DailyOps.Tech](#) · [OpsGate](#) · [FR](#)



Public : intégrateur, admin système, DSI / RSSI pilote

Version produit : 1.2 / lot V2 + capacités V3-P0 (Secure Rewrite, Risk, Shadow AI)

Maturité : V2 fonctionnel / pre-GA · V3-P0 en code - STATUS-V2.md · gap STATUS-GAP-V1-V3.md

Objectif : déployer OpsGate chez le client (pas seulement en lab dev)

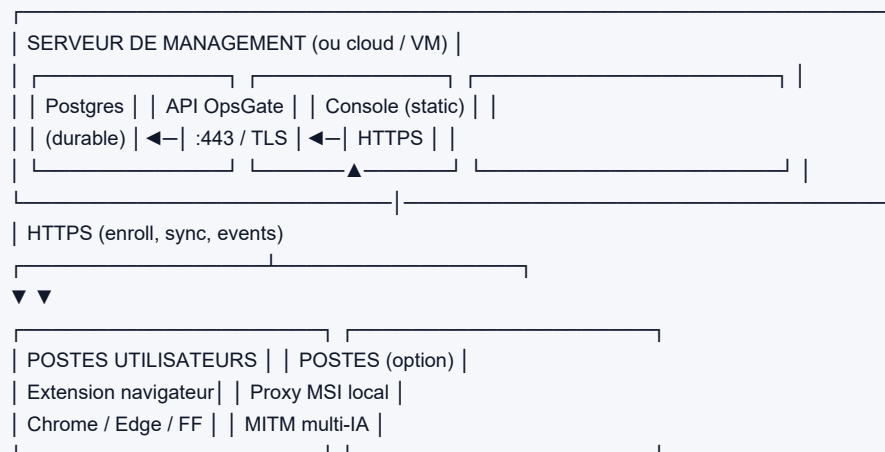
Ce document est le point d'entrée déploiement.

Les guides utilisateur décrivent l'usage de la console ; ici on décrit où installer quoi, dans quel ordre.

Document lié	Contenu
STATUS-V2.md	Avancement produit (livré vs suite)
PUBLICATION-STORES.md	Package & soumission CWS / AMO / Safari
FAQ-DEPLOIEMENT-V2.md	Questions MSI / licences / phases test
GUIDE-STACK-LOCALE.md	Lab Docker sur une machine de dev
architecture/CHROME-WEB-STORE-MDM.md	Force-install Chrome/Edge
architecture/FIREFOX-AMO.md	Firefox entreprise
architecture/PROXY-PROD-WINDOWS.md	Proxy MSI production
architecture/BACKUP.md	Sauvegardes
LICENCES-CLIENTS.md	Activation clé côté client

1. Vue d'ensemble - qui installe quoi

CHEZ LE CLIENT



Composant	Où	Obligatoire ?
Postgres	Serveur management	Oui en prod (sinon perte de données)
API	Serveur management	Oui pour mode org
Console	Serveur / CDN / reverse-proxy	Oui pour administrer
Extension	Chaque poste utilisateur	Oui (cœur produit)



Proxy local MSI

Postes (ou GPO)

Optionnel (filet HTTPS)

Règle d'or : l'extension ne parle jamais directement à Postgres. Elle parle uniquement à l'URL API (HTTPS en prod).

2. Scénarios de déploiement

Scénario	Description	Public
A. Lab mono-PC	Tout sur 127.0.0.1 (Docker + api:dev + console:dev + sideload)	Dev / démo
B. Pilote client (recommandé)	1 serveur API+DB + N postes extension (+ proxy optionnel)	Premier client
C. Production	TLS, secrets forts, MDM force-install, backups, SMTP	Déploiement durable

Ce guide détaille B puis les durcissements C. Pour le lab pur, voir aussi GUIDE-STACK-LOCALE.md.

3. Prérequis

3.1 Serveur de management

- OS : Linux (recommandé) ou Windows Server / Windows 11 avec Docker
- Node.js 20+ et pnpm
- Docker (Postgres) ou Postgres managé déjà fourni
- Ports ouverts :
 - 5432 Postgres (idéalement non exposé sur Internet - localhost / réseau privé)
 - 8787 API (dev) -> en prod : 443 via reverse-proxy (nginx, Caddy, IIS...)
 - 5173 console dev -> en prod : fichiers statiques + HTTPS
- Accès sortant SMTP si e-mails (OTP, exports, alertes)

3.2 Postes utilisateurs

- Windows 10/11 (ou macOS / Linux pour extension seule)
- Chrome ou Edge (Firefox supporté)
- Droits admin uniquement pour MSI proxy / force-install GPO (pas pour l'usage quotidien de l'extension sideload de test)

3.3 Fournitures DailyOps / intégrateur

- Code organisation (ex. ACME-2026) ou seed DEMO pour pilote
- Clé de licence full si hors essai (OPS-XXXX-...)
- URL API publique HTTPS (ex. <https://opsgate.client.tld>)
- Optionnel : secrets SSO, SMTP, SIEM

4. Phase 1 - Control plane (serveur)



Étape 1.1 - Récupérer le code / package

```
# Depuis le monorepo fourni par DailyOps
cd ops-gate
pnpm install
```

Étape 1.2 - Postgres durable

```
docker compose up -d
docker compose ps
# Conteneur opsgate-postgres healthy · port 5432
```

Production : utiliser un Postgres managé (RDS, Azure PG, etc.) et une DATABASE_URL dédiée (user fort, TLS).

Étape 1.3 - Variables d'environnement API

Créer un fichier .env (jamais commité) à partir de .env.example :

```
# Obligatoire prod
DATABASE_URL=postgres://USER:PASS@HOST:5432/opsgate
NODE_ENV=production
PORT=8787

# Admin initial (à changer à la 1re connexion)
OPSGATE_SETUP_EMAIL=admin@client.tld
OPSGATE_SETUP_PASSWORD=MotDePasseFort>=8car

# Break-glass offline (>= 2 h sans sync)
OPSGATE_VENDOR_RECOVERY=SecretLongEtUnique...

# Optionnel mais recommandé
OPSGATE_SMTP_HOST=smtp.client.tld
OPSGATE_SMTP_PORT=587
OPSGATE_SMTP_USER=...
OPSGATE_SMTP_PASS=...
OPSGATE_SMTP_FROM=OpsGate <noreply@client.tld>

# URL publiques (SSO, passkeys, liens mails)
OPSGATE_CONSOLE_URL=https://console.client.tld
OPSGATE_API_PUBLIC_URL=https://api.client.tld
```

Sans DATABASE_URL -> store mémoire : tout est perdu au redémarrage. À bannir chez le client.

Étape 1.4 - Démarrer l'API

```
# Charger .env ou exporter DATABASE_URL
pnpm --filter @opsgate/api start
# ou en lab : pnpm api:dev
```



Vérifier :

```
curl https://api.client.tld/health
# { "ok": true, "store": "postgres", "version": "1.2.0", ... }
```

Attendu : store=postgres.

Étape 1.5 - Console admin

Lab :

```
pnpm console:dev
# -> http://127.0.0.1:5173
```

Production :

```
pnpm --filter @opsgate/console build
# Servir packages/console/dist derrière HTTPS
# (nginx, IIS, Caddy, CloudFront...)
```

Configurer l'URL API dans la console (champ API ou paramètre build) pour pointer vers <https://api.client.tld>.

Étape 1.6 - Reverse-proxy TLS (prod)

Exemple minimal (concept) :

- <https://api.client.tld> -> <http://127.0.0.1:8787>
- <https://console.client.tld> -> fichiers statiques console

Certificats : Let's Encrypt / PKI client.

5. Phase 2 - Première configuration métier

1. Ouvrir la console -> se connecter (OPSGATE_SETUP_EMAIL / mdp).
2. Changer le mot de passe immédiatement.
3. Activer MFA TOTP (Paramètres -> Général) - obligatoire si multi-org.
4. Noter le code organisation (ex. affiché en Paramètres / licence).
5. (Option) Activer licence full : coller la clé OPS-... fournie par DailyOps.
6. Définir policy (action, sites IA) -> Force sync.
7. Configurer SMTP (Paramètres -> E-mail) + test d'envoi.
8. (Option) Notifications, SIEM, export planifié, backup config.

Checklist « serveur prêt » :

- [] /health -> store=postgres
- [] Login console OK + mdp change
- [] Code org communiqué aux postes
- [] HTTPS joignable depuis un poste utilisateur
- [] Backup Postgres planifié (scripts/backup-db.ps1 ou pg_dump)



6. Phase 3 - Extension sur les postes

6.1 Pilote (sideload, 1-20 postes)

Chrome / Edge

```
# Sur machine de build
pnpm build:chrome
# Dossier : build/chrome-mv3-prod
```

Sur le poste :

1. chrome://extensions ou edge://extensions
2. Mode développeur = ON
3. Charger l'extension non empaquetée -> dossier chrome-mv3-prod (ou ZIP fourni par l'intégrateur)

Firefox

```
pnpm build:firefox
# about:debugging -> charger build/firefox-mv3-prod/manifest.json
```

6.2 Production (force-install)

Navigateur	Méthode
Chrome / Edge	Chrome Web Store (unlisted) + MDM / GPO ExtensionInstallForcelist
Firefox	AMO + policies.json

```
pnpm store:all # -> dist/chrome-store/ + firefox-ammo/ + safari-store/ + STORES-INDEX.md
pnpm store:chrome # CWS seul
pnpm store:firefox # AMO seul
# Guide : docs/PUBLICATION-STORES.md
```

Détail policies :

architecture/CHROME-WEB-STORE-MDM.md ·

architecture/FIREFOX-AMO.md

Fichiers utiles dans le package store :

- dist/chrome-store/mdm/chrome-force-install.reg
- dist/chrome-store/mdm/edge-force-install.reg
- dist/chrome-store/mdm/intune-settings-catalog.json

6.3 Enrôlement org (chaque poste / profil)

1. Ouvrir Options de l'extension OpsGate.
2. URL API = https://api.client.tld (pas 127.0.0.1 sauf lab mono-PC).
3. Code organisation = code fourni.



4. Label appareil (ex. PC-FIN-12).
5. Enrôler.

Attendu : mode géré par l'org, policy reçue, agent visible dans Console -> Agents (<= 2 min ou Force sync).

6.4 Test de fumée poste

1. Aller sur un site IA (ex. ChatGPT).
2. Coller un secret de test (ex. clé factice sk-test...).
3. Le bandeau OpsGate apparaît (mask / block selon policy).
4. Console -> Événements : une ligne apparaît.

7. Phase 4 - Proxy local (optionnel)

Complète l'extension (filet si le DOM ne capture pas tout le trafic).

7.1 Build / install MSI

```
pnpm proxy:package # ou pipeline MSI
# Artefact : dist/opsgate-proxy-*.msi
```

Sur le poste (admin) :

```
msiexec /i dist/opsgate-proxy-1.2.0.msi /qn
```

- Binaires : C:/Program Files/OpsGate/Proxy/
- Data : %ProgramData%/OpsGate/Proxy/

7.2 Enroll proxy + CA

Le post-install peut automatiser CA + tâche planifiée. Sinon :

1. Générer / déployer le CA proxy (trust Root utilisateur ou machine).
2. Enrôler le proxy vers la même API + code org.
3. Configurer PAC / GPO pour le trafic HTTPS multi-IA (voir PROXY-PROD-WINDOWS.md).

Important : proxy ≠ bandeau orange. Le banner UI reste le rôle de l'extension.

8. Ordre de mise en service (résumé)

1. Postgres UP
2. API UP (store=postgres) + TLS
3. Console UP + 1er admin + mdp + MFA
4. Policy + licence
5. Extension sur 1 poste pilote + enroll
6. Test secret -> event en console
7. Extension flotte (MDM)
8. (Option) Proxy MSI
9. SMTP / alertes / SIEM / backups



9. Réseau & firewall

Flux	Source	Destination	Port
Enroll / sync / events	Postes (extension, proxy)	API client	443 (HTTPS)
Console admins	Postes admin	Console	443
API -> Postgres	Serveur API	DB	5432 (réseau privé)
API -> SMTP	Serveur API	SMTP	587 / 465
API -> SIEM	Serveur API	Syslog	UDP/TCP configuré

Ne pas exposer Postgres sur Internet.

10. Sécurité production (checklist)

- ☐ NODE_ENV=production
- ☐ DATABASE_URL + mdp forts
- ☐ OPSGATE_SETUP_PASSWORD et OPSGATE_VENDOR_RECOVERY non faibles
- ☐ TLS sur API et console
- ☐ MFA admin activée
- ☐ Pas de OPSGATE_ALLOW_DEV_ADMIN
- ☐ Backup Postgres quotidien + test restore
- ☐ Export config org avant upgrade (Paramètres -> Backup)
- ☐ Rotation secrets documentée (architecture/SECRET-ROTATION.md)

11. Validation « go client »

#	Test	OK
1	GET /health -> store=postgres	☐
2	Login console + changement mdp	☐
3	Agent enrôle visible	☐
4	Force sync applique un changement policy	☐
5	Event de detection en console	☐
6	Redemarrage API conserve agents / admins	☐
7	Backup pg_dump ou script	☐

Automatisable (lab) :

```
# DATABASE_URL=...
pnpm api:validate-pg
```




12. Incidents fréquents

Symptôme	Cause probable	Action
Extension reste local_only	Mauvaise URL API / code org	Vérifier Options + /health depuis le poste
Events vides	Pas d'enroll / eventReporting off	Enroll + policy + test secret
Login impossible	Mauvais mdp / compte verrouillé	Principal déverrouille ; OTP SMTP
Perte de données au restart	API en memory	DATABASE_URL + restart
Pack verify failed	Clés signature / API down	Vérifier API et clés ed25519
MSI proxy sans effet	Pas de PAC / CA non trustée	GPO PAC + store Root

13. Qui fait quoi (RACI simplifié)

Tâche	Intégrateur / DailyOps	IT client	Utilisateur final
Serveur API + DB	X	(support)	
TLS / DNS	(support)	X	
MDM force-install	(support)	X	
Policy / licences	X	X	
Enroll poste (sideload pilote)	X	(support)	
Usage quotidien IA			X

14. Après le déploiement

- Former les admins : GUIDE-UTILISATEUR-V2.md
- Brief décideurs : DECIDEURS-V2-FR.md
- FAQ technique : FAQ-DEPLOIEMENT-V2.md
- Catalogue capacités : architecture/BACKEND-V2-CATALOG.md
- Différenciation V3 (Rewrite / Risk / Shadow) : RELEASE-v3.md
- Langue des agents (banner) : Console -> Paramètres -> Langue des agents (fr / en / auto) puis sync
- Republier le pack de règles si l'engine a évolué (IBAN, etc.) : console Packs ou script scripts/republish-engine-pack.mjs

Support : contact@dailyops.tech - indiquer code org, version API (/health), et navigateur.

OpsGate · Guide déploiement client · DailyOps.Tech · 18 juillet 2026